



**Universidad
Tecnológica
del Perú**

UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Sanipatitas - Veterinaria HC

Curso

Arquitectura Orientada al Servicio

Sección

47007

Integrantes

Alcántara Paico, Luis Sebastián	U22218459
Contreras Delgado, Jorge Renzo Paolo	U22202038
Huamán Domador, Luis Alberto	U21213445
Ramos Yampufe, Martin Alexander	U22214724
Ruiz Cardoza, Richard	U20229354

Docente

Minguillo Rubio, Cesar Augusto

Chiclayo, Perú

2026

Índice

Introducción	7
Diagnóstico del negocio y procesos	8
Descripción general del negocio	8
Proceso principal que genera valor	8
Mapa de procesos actual	10
AS-IS	10
TO-BE	11
Flujo de información entre áreas	11
Identificación de puntos de dolor	12
Problemas y cuellos de botella en los procesos	13
Objetivos del proyecto SOA	13
Objetivo general	13
Objetivos específicos	13
Formulación del problema	14
Inventario tecnológico (sistemas actuales)	15
Registros y recursos usados actualmente	15
Registros físicos y mecanismos manuales	15
Bases de datos existentes	15
Lenguajes de programación y plataformas	16
Comunicación actual de la información	17
Definición de requerimientos del servicio	18
Requerimientos funcionales	18
Requerimientos no funcionales	22
Requerimientos de seguridad	26
Requerimientos de disponibilidad y rendimiento	26
Interoperabilidad con sistemas externos	26

	3
Gobernanza y roles	28
Identificación de stakeholders	28
Roles y responsabilidades	28
Reglas de negocio	29
Levantamiento de información (entrevistas)	30
Entrevista a la responsable del negocio	30
Análisis de resultados de la entrevista	30
Identificación de servicios candidatos (SOA)	32
Funciones del negocio candidatas a servicios	32
Identificación de servicios compartidos	32
Definición preliminar de servicios	33
Diseño de inventario de servicios	34
Arquitectura empresarial y capas SOA	36
Capa de negocio	36
Capa de aplicación	36
Capa de datos	36
Capa de tecnología	37
Composición de servicios y Enterprise Service Bus	38
Orquestación de servicios en el sistema	39
Flujo principal: atención integral del paciente	39
Flujo secundario: control de inventario	40
Manejo de errores	40
Despliegue orquestado con Docker y k3s	40
Implementación Backend	42
Servicios del sistema	42
API REST y base de datos	42
Comunicación entre servicios	43

Integración Frontend	43
Stack del cliente	43
Consumo de servicios mediante cliente generado	44
Flujos de usuario principales	45
Administración de Procesos y Auditoría	45
Servicio transversal de auditoría	45
Modelado BPMN con Camunda 8	48
Seguridad Implementada	49
Autenticación con JWT (ES256)	49
Roles y autorización	51
Propagación del token entre servicios	51
Seguridad en el frontend	52
Conclusiones	53
Bibliografía	55

Tablas

Tabla 1	Requerimiento Funcional RF001: Almacenamiento de información de pacientes .	18
Tabla 2	Requerimiento Funcional RF002: Almacenamiento de Información de Productos .	18
Tabla 3	Requerimiento Funcional RF003: Registro y Gestión de Citas	19
Tabla 4	Requerimiento Funcional RF004: Registro de Clientes	20
Tabla 5	Requerimiento Funcional RF005: Registro de Ventas	20
Tabla 6	Requerimiento Funcional RF006: Búsqueda de mascota por código QR	21
Tabla 7	Requerimiento No Funcional RNF001: Tolerancia a fallos	22
Tabla 8	Requerimiento No Funcional RNF002: Interfaz intuitiva y fácil de usar	22
Tabla 9	Requerimiento No Funcional RNF003: Eficiencia y tiempo de respuesta	23
Tabla 10	Requerimiento No Funcional RNF004: Modularidad del Código	23
Tabla 11	Requerimiento de Plataforma RP001: Compatibilidad con navegadores	24
Tabla 12	Requerimiento de Seguridad RS001: Autenticación de los usuarios	24
Tabla 13	Requerimiento Técnico RT001: Plataforma de desarrollo	25
Tabla 14	Inventario de servicios del sistema	34
Tabla 15	Inventario de servicios del sistema	42
Tabla 16	Endpoints principales por servicio	42
Tabla 17	Stack tecnológico del frontend	43
Tabla 18	Tablas de auditoría generadas por Hibernate Envers	46
Tabla 19	Mecanismos de envío de logs a Loki por servicio	47
Tabla 20	Roles del sistema y sus permisos	51

Figuras

Figura 1 Diagrama AS-IS del mapa de procesos actual	10
Figura 2 Diagrama TO-BE del mapa de procesos propuesto	11
Figura 3 Diagrama BPMN del flujo de auditoría de transacción crítica	48
Figura 4 Diagrama BPMN del flujo de aprobación de cita veterinaria	48
Figura 5 Diagrama BPMN del flujo de alerta de inventario por venta	49
Figura 6 Diagrama BPMN del flujo de registro de paciente	49
Figura 7 Diagrama BPMN del flujo de autenticación y autorización	49

Introducción

En la actualidad, las organizaciones buscan optimizar sus procesos mediante el uso de tecnologías que permitan integrar la información y mejorar la eficiencia operativa. En este contexto, la arquitectura orientada a servicios (SOA) surge como una alternativa que facilita la interoperabilidad entre sistemas y la reutilización de funcionalidades.

El presente proyecto se desarrolla en el contexto de la Clínica Veterinaria HC, ubicada en la ciudad de Ferreñafe, provincia de Ferreñafe, región Lambayeque, Perú. Esta clínica opera en un entorno donde la demanda de atención a mascotas y venta de productos relacionados es constante y creciente. Sin embargo, al igual que otras veterinarias de la localidad, no cuenta con un sistema que le permita automatizar sus procesos internos, lo que la coloca en desventaja operativa frente a las exigencias del mercado actual.

Desde el punto de vista sociocultural, los clientes muestran una clara preferencia por atenciones eficaces y rápidas, y actualmente llegan a la clínica principalmente a través de sus redes sociales. Esto indica que, si bien existe un canal de captación activo, la experiencia una vez dentro del establecimiento aún depende de procesos manuales y desconectados que pueden afectar la percepción del servicio.

En cuanto al entorno tecnológico del rubro, la ausencia de herramientas digitales integradas como sistemas de historial clínico, gestión de citas o control de inventario representa una brecha significativa que limita la calidad del servicio ofrecido y la capacidad de crecimiento del negocio.

Ante esta problemática, se plantea el análisis del negocio desde un enfoque SOA para identificar los procesos clave, detectar puntos críticos y definir oportunidades de mejora mediante la digitalización de registros que hoy se manejan en papel, cuadernos y mensajería por WhatsApp.

Diagnóstico del negocio y procesos

Descripción general del negocio

La empresa objeto de estudio es una clínica veterinaria dedicada a la atención médica de mascotas, así como a la comercialización de productos relacionados con su cuidado. Se encuentra ubicada en la provincia de Ferreñafe, en la región Lambayeque, y atiende una demanda constante de clientes que requieren servicios veterinarios y productos especializados.

Actualmente, la clínica no cuenta con un sistema digital ni con una base de datos que permita gestionar de manera eficiente la información. Los historiales clínicos, las citas y el control de inventarios se registran en papel, cuadernos y fichas físicas, lo que dificulta su consulta, actualización y seguimiento. Asimismo, la comunicación con los clientes se realiza de forma directa o mediante medios informales, sin una plataforma interna que centralice la información generada durante la atención.

La clínica gestiona sus procesos de forma manual o con herramientas desconectadas entre sí, como cuadernos, hojas de cálculo y mensajería por WhatsApp. Esta situación no es aislada: los problemas que normalmente se presentan en las clínicas veterinarias suelen ser el inadecuado control en los registros médicos de las mascotas, pues al registrar manualmente la información existe la probabilidad de que estos datos se pierdan, generando inconvenientes con los clientes (Ochoa et al., 2021).

Proceso principal que genera valor

El proceso principal que genera valor en la organización es la atención integral de pacientes veterinarios, el cual comprende desde la llegada del cliente hasta la finalización del servicio. Actualmente, este flujo se gestiona de manera manual, con registros físicos y coordinación directa por parte de la jefa veterinaria.

Este proceso incluye actividades como el registro del cliente y del paciente, la evaluación médica del animal, la consulta del historial clínico, el diagnóstico y tratamiento, el registro de la atención, la programación de citas futuras y la venta de productos relacionados.

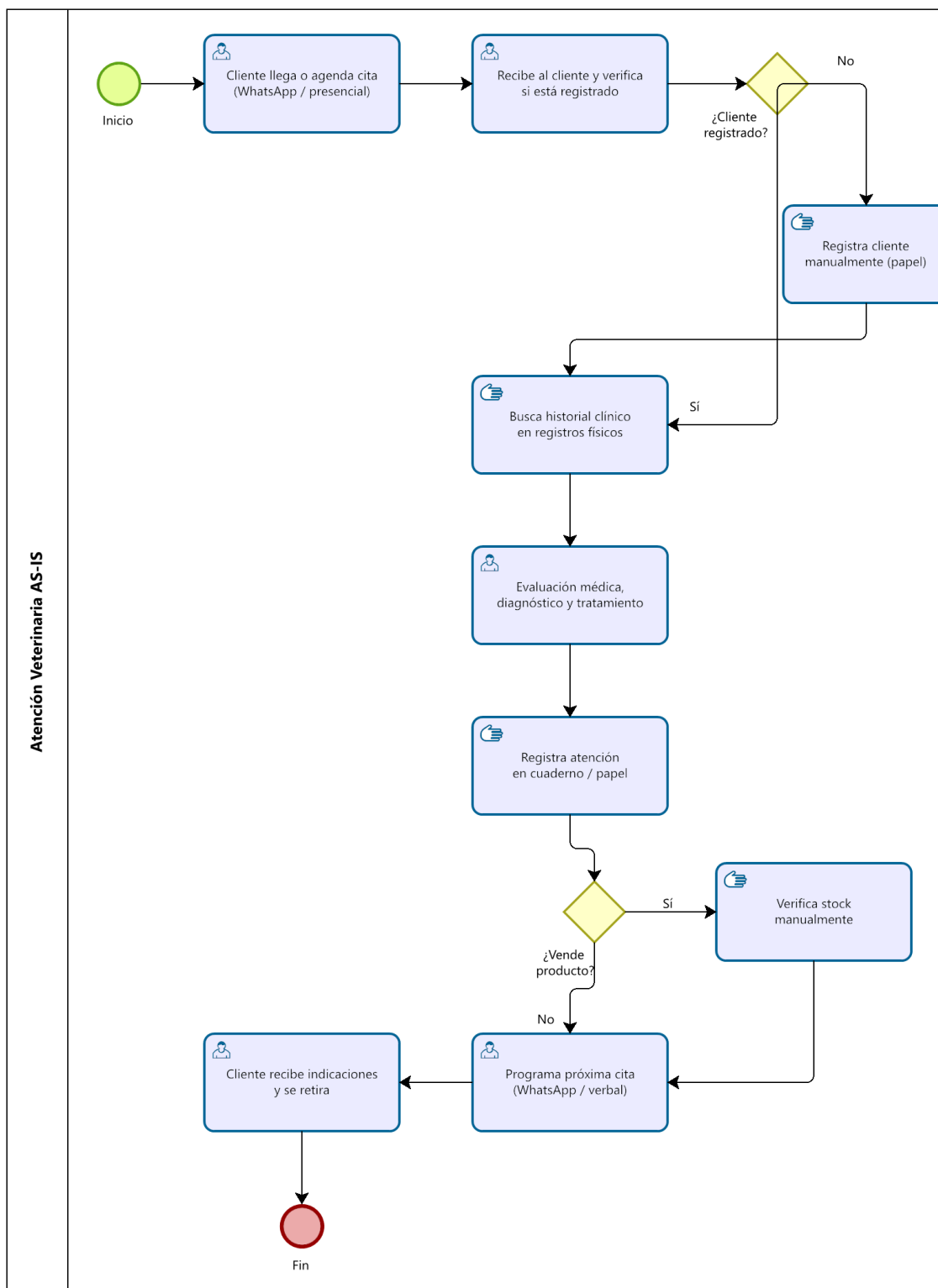
Este flujo es fundamental, ya que impacta directamente en la satisfacción del cliente y en la calidad del servicio brindado.

Mapa de procesos actual

AS-IS

Figura 1

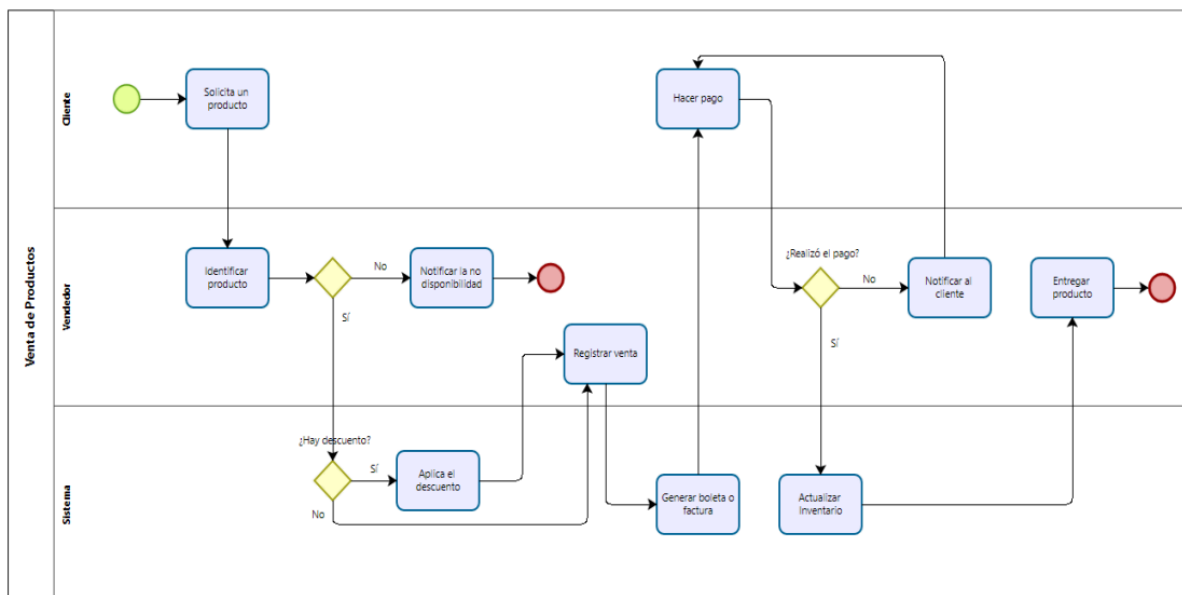
Diagrama AS-IS del mapa de procesos actual



TO-BE

Figura 2

Diagrama TO-BE del mapa de procesos propuesto



Flujo de información entre áreas

El flujo de información dentro de la clínica veterinaria se desarrolla principalmente de manera manual o mediante registros aislados. La información del cliente y del paciente es gestionada directamente por la jefa veterinaria, quien centraliza las actividades de atención, registro y seguimiento.

Uno de los problemas más evidentes es la fragmentación de la información. Los datos del paciente, el inventario, los pagos y los historiales clínicos se encuentran dispersos en distintos soportes sin ninguna conexión entre sí, lo que obliga al personal a consultar múltiples fuentes para completar una sola tarea. En el sector sanitario, la información fragmentada y dispersa en diferentes sistemas dificulta la recopilación y el análisis de los datos, lo que en términos de gestión puede derivar en ineficiencias en el control de costos y el aprovechamiento de los recursos disponibles («Desafíos del acceso a la información en salud: cuáles son y cómo resolverlos», 2023).

Asimismo, la información relacionada con citas, historial clínico y ventas no se encuentra centralizada, lo que dificulta el acceso rápido a los datos y genera dependencia de

registros individuales. Esto ocasiona que la información no fluya de manera eficiente entre las distintas funciones del negocio, limitando la capacidad de respuesta ante las necesidades del cliente.

Identificación de puntos de dolor

A partir del análisis del negocio se identificaron diversos puntos críticos. La falta de un registro centralizado provoca que los datos se encuentren dispersos en cuadernos, hojas de cálculo y mensajes de WhatsApp, sin ninguna conexión entre sí. El control de stock de medicamentos e insumos depende de revisiones físicas o de la memoria del personal. Sin un sistema adecuado, las clínicas pueden enfrentar problemas de falta de stock, sobreabastecimiento o desperdicio de recursos, lo que afecta negativamente su estabilidad financiera («Estrategias para una gestión eficiente de inventario en clínicas veterinarias», 2024).

La ausencia de un historial clínico centralizado impide hacer seguimientos post-consulta, identificar patrones de enfermedad recurrente o generar reportes útiles para la toma de decisiones clínicas y administrativas. El historial clínico electrónico se ha convertido en el estándar de atención en medicina veterinaria, permitiendo a los profesionales acceder a toda la información médica de forma rápida y mejorando la precisión del diagnóstico (Marks, 2024).

Los cobros mal registrados, las deudas no rastreadas y la falta de reportes económicos claros representan un riesgo constante para la estabilidad del negocio. Este panorama es representativo de una brecha más amplia en el contexto peruano: el 73% de las pymes reconoce no saber qué herramientas tecnológicas utilizar ni contar con el talento adecuado («Digitalización en pausa: el 73% de pymes no sabe por dónde empezar su transformación tecnológica», 2025). A ello se suman la dificultad para localizar historiales mediante búsqueda manual en cuadernos y fichas, la duplicidad de registros de pacientes o clientes por falta de un sistema unificado, y la dependencia de una sola persona que centraliza todas las funciones operativas, generando cuellos de botella y limitando la escalabilidad del negocio.

Problemas y cuellos de botella en los procesos

Los principales cuellos de botella identificados en los procesos incluyen la búsqueda manual de información en cuadernos y fichas, la duplicidad de registros de pacientes o clientes, la falta de coordinación entre la atención médica y la gestión de citas, las dificultades en el control de stock de productos y los posibles errores de transcripción al registrar datos.

A esto se suma la ausencia de un historial clínico centralizado, que impide hacer seguimientos post-consulta, identificar patrones de enfermedad recurrente o generar reportes útiles para la toma de decisiones clínicas y administrativas.

En conjunto, estas deficiencias operativas limitan severamente la capacidad de la Clínica Veterinaria HC para ofrecer un servicio de calidad y sostenerse competitivamente.

Objetivos del proyecto SOA

Objetivo general

Desarrollar e implementar un sistema web integrado basado en una arquitectura orientada a servicios para la Clínica Veterinaria HC que centralice y automatice los procesos clínicos y administrativos, tales como la gestión de historiales médicos, citas, inventario y ventas, con el fin de mejorar la calidad del servicio, optimizar los tiempos de atención y fortalecer la eficiencia operativa de la organización.

Objetivos específicos

Implementar un módulo de gestión de historiales clínicos que permita registrar, almacenar y consultar de forma estructurada la información médica de las mascotas, incluyendo diagnósticos, tratamientos, vacunas y cirugías, garantizando la trazabilidad de cada atención.

Desarrollar un módulo de gestión de citas que permita programar, modificar y visualizar las consultas médicas, optimizando la planificación de la atención y reduciendo los tiempos de espera de los clientes.

Diseñar un módulo de gestión de inventario y ventas que permita controlar el stock de productos, registrar transacciones, generar alertas de reposición y gestionar fechas de vencimiento, asegurando la disponibilidad de insumos y un adecuado control comercial.

Implementar un sistema de generación y lectura de códigos QR que permita a la veterinaria identificar y buscar rápidamente a cada mascota mediante el escaneo de su código QR único asociado, utilizando el protocolo sanipatitas://patient/{uuid}.

Formulación del problema

¿De qué manera la implementación de un sistema web integrado basado en una arquitectura orientada a servicios permitirá optimizar la gestión de los procesos clínicos y administrativos de la Clínica Veterinaria HC, mejorando la calidad de atención y la eficiencia operativa del negocio?

Inventario tecnológico (sistemas actuales)

Registros y recursos usados actualmente

Actualmente, la clínica veterinaria no cuenta con un sistema informático ni con una base de datos que permita gestionar sus procesos operativos. Las actividades principales, como el registro de pacientes, la programación de citas y el control de productos, se realizan en cuadernos, fichas y hojas de control, de forma manual y sin herramientas especializadas.

Esta situación genera una gestión fragmentada de la información, donde los datos no se encuentran centralizados, lo que dificulta su acceso y actualización en tiempo real. Asimismo, la comunicación con los clientes se realiza de forma directa o mediante medios informales, sin un repositorio interno que permita organizar la información generada durante la atención.

Registros físicos y mecanismos manuales

En la clínica veterinaria no se identifican sistemas heredados digitales. En su lugar, se utilizan cuadernos, fichas, hojas de control y anotaciones manuscritas para registrar la atención, las citas, los historiales clínicos y el inventario. Estos soportes físicos conservan información valiosa del negocio, pero presentan limitaciones para su consulta, actualización, trazabilidad y escalabilidad.

Bases de datos existentes

Actualmente, la empresa no cuenta con una base de datos estructurada que permita almacenar y gestionar la información de manera centralizada.

La información relacionada con pacientes, citas, ventas y productos se mantiene en registros físicos dispersos, lo que dificulta su consulta, actualización y análisis. Esto genera dependencia de anotaciones manuales y aumenta el riesgo de errores, omisiones o pérdida de información.

Por ello, se identifica la necesidad de implementar una base de datos relacional (PostgreSQL 17) que permita estructurar y compartir los datos del negocio de manera eficiente, con replicación en streaming para garantizar la disponibilidad de los datos.

Lenguajes de programación y plataformas

Debido a que la clínica no cuenta con un sistema implementado ni con infraestructura digital de gestión, actualmente no se utilizan lenguajes de programación ni plataformas tecnológicas específicas para la gestión de procesos.

Para la implementación del sistema basado en SOA se ha definido una arquitectura tecnológica cuyo backend se divide en servicios independientes que se comunican mediante JWT. El Auth Service está implementado en TypeScript sobre Bun 1.x con el framework Elysia 1.4 y la librería Better Auth 1.6.10 para gestión de autenticación y sesiones en Redis. Los servicios de negocio están implementados en Java 25 con Quarkus 3.35.2: Patient Service gestiona pacientes, clientes y catálogos de especies y razas, Appointment Service gestiona citas, agenda y disponibilidad de horarios, MedicalRecord Service gestiona el historial clínico siguiendo los recursos del estándar HL7 FHIR, Inventory Service gestiona productos, stock, alertas de reposición y movimientos de inventario, Sales Service gestiona ventas, ítems de venta y comprobantes, Prescription Service gestiona recetas médicas y sus ítems.

El frontend se desarrolla con Astro y React, utilizando TypeScript y generando el cliente API automáticamente desde las especificaciones OpenAPI mediante `@hey-api/openapi-ts`. La funcionalidad de códigos QR se implementa como una característica del frontend: cada mascota registrada tiene un código QR asociado con el protocolo `sanipatitas://patient/{uuid}`, generado desde el navegador, que permite su identificación rápida mediante escaneo. La base de datos es PostgreSQL 17 con replicación en streaming asíncrona y migraciones gestionadas con Drizzle Kit 0.31. Redis 7 se utiliza para almacenamiento de sesiones activas y caché de consultas frecuentes.

Estas tecnologías permitirán la construcción de servicios reutilizables dentro de una arquitectura orientada a servicios, cumpliendo con el requisito de que más del 50% del código backend esté en Java.

Comunicación actual de la información

En el estado actual no existen mecanismos de comunicación entre sistemas, ya que la clínica no dispone de plataformas tecnológicas integradas.

La información se transmite de forma verbal y mediante registros en papel, dependiendo principalmente de la intervención directa del personal. Esto genera retrasos, duplicidad de datos y falta de sincronización entre las actividades del negocio.

Para la solución propuesta se plantea implementar mecanismos de comunicación basados en servicios RESTful con autenticación JWT, permitiendo el intercambio de información de forma automatizada y en tiempo real. Cada servicio expone su esquema OpenAPI para documentación y generación automática de clientes.

Definición de requerimientos del servicio

Requerimientos funcionales

Tabla 1

Requerimiento Funcional RF001: Almacenamiento de información de pacientes

RF001	Almacenamiento de información de pacientes
Versión	1.0 (Fecha: 2026-04-18)
Dependencias	Patient Service para el CRUD de pacientes, Appointment Service para la gestión de citas y registro de clientes.
Descripción	El sistema deberá almacenar la información correspondiente a los pacientes, incluyendo la identificación del paciente con nombre, especie, raza, edad, peso y color, el historial médico completo con consultas anteriores, diagnósticos, tratamientos, vacunaciones, procedimientos y observaciones, la información de contacto del propietario, el registro de citas con fechas, motivos y resultados de consultas, y un código QR único con protocolo sanipatitas://patient/{uuid} generado automáticamente para identificación rápida.
Importancia	Alta, ya que el mantenimiento de un historial clínico completo y accesible es crucial para el diagnóstico y tratamiento efectivo de los pacientes.
Prioridad	Alta, debido a su impacto en la calidad del cuidado de los pacientes y en la eficiencia operativa de la veterinaria.
Estado	En planificación.
Comentarios	Este requisito es fundamental para el funcionamiento del sistema, asegurando que toda la información relevante sobre los pacientes esté disponible y actualizada. Cada paciente tendrá un código QR único con el protocolo sanipatitas://patient/{uuid} que permitirá su identificación rápida mediante escaneo.

Tabla 2

Requerimiento Funcional RF002: Almacenamiento de Información de Productos

RF002	Almacenamiento de Información de Productos
Versión	1.0 (Fecha: 2026-04-18)
Dependencias	Inventory Service para la gestión de inventario.

RF002	Almacenamiento de Información de Productos
Descripción	El sistema deberá almacenar la información correspondiente a los productos disponibles en la veterinaria, incluyendo nombre del producto, descripción, categoría entre medicamento, alimento, accesorio o higiene, precio unitario, stock actual y stock mínimo, fecha de vencimiento si aplica, y estado activo o inactivo.
Importancia	Alta, ya que una gestión precisa del inventario es esencial para asegurar la disponibilidad de productos y evitar pérdidas por desabastecimiento o expiración.
Prioridad	Alta, para garantizar que la veterinaria pueda mantener un control efectivo del inventario y satisfacer la demanda de los clientes.
Estado	En planificación.
Comentarios	Este requisito es crucial para la eficiencia operativa de la veterinaria y la satisfacción del cliente al asegurar que los productos estén disponibles cuando se necesiten. El sistema generará alertas automáticas cuando el stock alcance el nivel mínimo.

Tabla 3

Requerimiento Funcional RF003: Registro y Gestión de Citas

RF003	Registro y Gestión de Citas
Versión	1.0 (Fecha: 2026-04-18)
Dependencias	Appointment Service para la gestión de citas, MedicalRecord Service para el historial clínico y Patient Service para el registro de pacientes.
Descripción	El sistema deberá almacenar y gestionar la información correspondiente a las citas médicas agendadas para los pacientes, incluyendo fecha y hora de la cita, fecha de creación, paciente asociado, cliente propietario asociado, veterinario asignado, estado entre programada, en progreso, completada o cancelada, motivo de la consulta y notas adicionales.
Importancia	Alta, para asegurar una organización efectiva del tiempo del veterinario y proporcionar un buen servicio al cliente.
Prioridad	Alta, debido a su impacto en la planificación y gestión de las consultas médicas.
Estado	En planificación.

RF003	Registro y Gestión de Citas
Comentarios	Este requisito es esencial para la gestión eficiente de las citas y la programación de consultas, impactando directamente en la calidad del servicio ofrecido. El sistema debe evitar la superposición de horarios.

Tabla 4*Requerimiento Funcional RF004: Registro de Clientes*

RF004	Registro de Clientes
Versión	1.0 (Fecha: 2026-04-18)
Dependencias	No depende de otros servicios.
Descripción	El sistema tendrá la opción de registrar los clientes para contar con su información de contacto, además de poder asociarlos con sus mascotas. Los datos específicos incluyen nombres y apellidos, número de identificación, tipo de identificación, dirección, teléfonos y correo electrónico.
Importancia	Alta, puesto que ayudará a que la veterinaria pueda tener un contacto directo con los clientes para seguimientos, recordatorios de citas y campañas de vacunación.
Prioridad	Media, no es del todo necesario pero representa una gran ventaja.
Estado	En planificación.
Comentarios	Puede significar un beneficio para la veterinaria.

Tabla 5*Requerimiento Funcional RF005: Registro de Ventas*

RF005	Registro de Ventas
Versión	1.0 (Fecha: 2026-04-18)
Dependencias	Sales Service para el registro de ventas, Inventory Service para la información de productos y Patient Service para el registro de clientes.
Descripción	El sistema debe almacenar las ventas realizadas por los clientes, registrando tanto los detalles de la transacción como los productos involucrados. Los datos de la venta incluyen cliente asociado, usuario vendedor, fecha y hora, total, método de pago entre efectivo, tarjeta o transferencia, y estado entre completada o cancelada. Los ítems de venta incluyen producto vendido, cantidad, precio unitario y subtotal.

RF005	Registro de Ventas
Importancia	Alta, ya que será necesario tener un control de las ventas de los productos de forma automática y actualizar el inventario consecuentemente.
Prioridad	Alta, puesto que le permitirá conocer la situación actual de los productos de la veterinaria y generar reportes económicos.
Estado	En planificación.
Comentarios	Este requisito es importante para mantener un control de las ventas dentro de la veterinaria y actualizar automáticamente el stock de productos.

Tabla 6

Requerimiento Funcional RF006: Búsqueda de mascota por código QR

RF006	Búsqueda de mascota por código QR
Versión	1.0 (Fecha: 2026-05-18)
Dependencias	Patient Service para la información de pacientes. La generación y lectura del código QR se implementa como funcionalidad del frontend.
Descripción	El sistema deberá generar automáticamente un código QR único para cada mascota registrada utilizando el protocolo <code>sanipatitas://patient/{uuid}</code> , donde {uuid} es el identificador único del paciente. Este QR permitirá la búsqueda e identificación rápida del paciente mediante escaneo. Al leer el código, el sistema extraerá el UUID de la URI y mostrará inmediatamente la ficha del paciente con su historial clínico completo. Los datos específicos incluyen el código QR con URI <code>sanipatitas://patient/{uuid-paciente}</code> generado automáticamente, la asociación del QR con el identificador del paciente en la base de datos, la capacidad de escaneo mediante cámara del dispositivo o lector QR, la validación del protocolo <code>sanipatitas://patient/</code> antes de procesar la búsqueda, la redirección automática a la ficha del paciente tras el escaneo, y la opción de imprimir el QR en formato de etiqueta o tarjeta para el propietario.
Importancia	Alta, ya que permite a la veterinaria identificar rápidamente a cada mascota sin necesidad de buscar manualmente su registro, reduciendo significativamente los tiempos de atención y evitando errores de identificación, especialmente en situaciones de emergencia.

RF006	Búsqueda de mascota por código QR
Prioridad	Alta, puesto que agiliza el flujo de trabajo diario de la clínica y mejora la experiencia tanto del personal como del cliente.
Estado	En planificación.
Comentarios	El código QR se genera automáticamente al registrar una nueva mascota con el formato <code>sanipatitas://patient/{uuid}</code> . El propietario puede recibir una etiqueta impresa con el QR para colocar en la correa o collar de la mascota, facilitando su identificación en cualquier momento. El protocolo personalizado <code>sanipatitas://patient/</code> permite que la aplicación reconozca automáticamente los códigos QR del sistema y los diferencie de otros códigos QR externos.

Requerimientos no funcionales

Tabla 7

Requerimiento No Funcional RNF001: Tolerancia a fallos

RNF001	Tolerancia a fallos
Versión	1.0 (Fecha: 2026-04-18)
Dependencias	Ninguna.
Descripción	El sistema debe ser capaz de detectar y gestionar automáticamente cualquier fallo o error que pueda surgir para minimizar el impacto en la experiencia del usuario. La base de datos cuenta con replicación en streaming para garantizar la disponibilidad de los datos incluso ante fallos del servidor primario. Cada servicio es independiente, por lo que un fallo en uno no afecta a los demás.
Importancia	Alta.
Prioridad	Media.
Estado	En planificación.
Comentarios	La implementación de mecanismos de tolerancia a fallos es esencial para garantizar la fiabilidad del sistema ante posibles situaciones adversas. El failover de la base de datos es manual: un operador puede promover la réplica a primario si el servidor principal falla.

Tabla 8

Requerimiento No Funcional RNF002: Interfaz intuitiva y fácil de usar

RNF002	Interfaz intuitiva y fácil de usar
Versión	1.0 (Fecha: 2026-04-18)
Dependencias	Ninguna.
Descripción	La interfaz de usuario del sistema debe ser intuitiva y fácil de usar para que la veterinaria pueda navegar sin problema alguno. Las vistas deben ser intuitivas y accesibles, facilitando la consulta rápida de información crítica como historiales clínicos, citas programadas y alertas de inventario.
Importancia	Alta.
Prioridad	Alta.
Estado	En planificación.
Comentarios	La usabilidad es fundamental para la adopción del sistema por parte de la veterinaria, especialmente considerando que el personal puede no tener experiencia previa con sistemas digitales.

Tabla 9

Requerimiento No Funcional RNF003: Eficiencia y tiempo de respuesta

RNF003	Eficiencia y tiempo de respuesta
Versión	1.0 (Fecha: 2026-04-18)
Dependencias	Ninguna.
Descripción	El sistema debe responder de manera rápida y ser eficiente con el fin de brindarle una rápida atención al cliente. Las consultas de historial clínico deben responder en menos de 500ms para garantizar una experiencia de uso fluida. La búsqueda por QR debe ser instantánea.
Importancia	Alta.
Prioridad	Alta.
Estado	En planificación.
Comentarios	La capacidad de respuesta inmediata del sistema contribuye significativamente a una experiencia de usuario fluida y eficiente. Se prevé implementar una capa de caché con Redis para optimizar las consultas frecuentes.

Tabla 10

Requerimiento No Funcional RNF004: Modularidad del Código

RNF004	Modularidad del Código
Versión	1.0 (Fecha: 2026-04-18)
Dependencias	Ninguna.
Descripción	El código del sistema debe estar organizado en servicios independientes y cohesivos, facilitando la comprensión, modificación y mantenimiento del mismo. La arquitectura de servicios (Auth Service, Patient Service, Appointment Service, MedicalRecord Service, Inventory Service, Sales Service, Prescription Service) refleja este principio, permitiendo que cada servicio evolucione de forma independiente.
Importancia	Alta.
Prioridad	Alta, puesto que define el desarrollo del sistema.
Estado	En desarrollo.
Comentarios	La modularidad del código permite una mayor flexibilidad y facilidad para realizar cambios y mejoras en el sistema a lo largo del tiempo.

Tabla 11

Requerimiento de Plataforma RP001: Compatibilidad con navegadores

RP001	Compatibilidad con navegadores
Versión	1.0 (Fecha: 2026-04-18)
Dependencias	Ninguna.
Descripción	Es importante que el sistema pueda ser usado independientemente del navegador de preferencia de la veterinaria. La aplicación web debe ser accesible desde cualquier dispositivo con conexión a internet.
Importancia	Alta, ya que le dará la mejor comodidad posible a la jefa veterinaria.
Prioridad	Alta, ya que definirá la comodidad del sistema.
Estado	En desarrollo.
Comentarios	La compatibilidad es importante, puesto que le permitirá agilizar el uso del sistema desde diferentes dispositivos y navegadores.

Tabla 12

Requerimiento de Seguridad RS001: Autenticación de los usuarios

RS001	Autenticación de los usuarios
Versión	1.0 (Fecha: 2026-04-18)
Dependencias	Ninguna.

RS001	Autenticación de los usuarios
Descripción	El sistema debe contar con un sistema de autenticación robusto basado en Better Auth 1.6.10, implementado sobre Elysia 1.4. Este módulo gestiona el inicio de sesión mediante email y contraseña, administra las sesiones activas almacenándolas en Redis, y genera JWTs firmados con el algoritmo ES256 basado en curva elíptica P-256. Además, expone un endpoint JWKS para que los demás servicios puedan validar los tokens de manera independiente.
Importancia	Alta, ya que proporcionará una capa sólida de seguridad para restringir el acceso al sistema.
Prioridad	Alta, ya que definirá la seguridad y confianza del sistema.
Estado	En desarrollo.
Comentarios	Es fundamental que exista este requisito de seguridad para evitar que el sistema sea vulnerado por terceros. Las contraseñas se almacenan utilizando el algoritmo bcrypt.

Tabla 13

Requerimiento Técnico RT001: Plataforma de desarrollo

RT001	Plataforma de desarrollo
Versión	1.0 (Fecha: 2026-04-18)
Dependencias	Ninguna.
Descripción	El sistema deberá respetar las siguientes restricciones técnicas: en el frontend se utilizarán tecnologías web como Astro, React y TypeScript, mientras que en el backend se empleará una arquitectura de servicios desacoplados con Auth Service en Bun, Elysia y Better Auth, y los servicios de negocio en Java 25 con Quarkus 3.35.2, incluyendo Patient, Appointment, MedicalRecord, Inventory, Sales, Prescription. La base de datos será PostgreSQL 17 con Redis 7 para caché y sesiones.
Importancia	Alta, puesto que esto será clave para definir el comportamiento del sistema.
Prioridad	Alta, puesto que define el desarrollo del sistema.
Estado	En desarrollo.
Comentarios	Esta restricción es fundamental puesto que determina las tecnologías que permitirán la escalabilidad y mantenimiento del sistema a futuro. Los servicios en Java representan más del 50% del código total del back-end.

Requerimientos de seguridad

El sistema debe garantizar la protección de la información mediante múltiples mecanismos de seguridad. La autenticación se realiza mediante Better Auth 1.6.10 con JWT firmados mediante ES256, sesiones almacenadas en Redis y endpoint JWKS para validación autónoma de tokens por parte de cada servicio. La autorización se basa en un sistema de roles diferenciados donde el administrador posee acceso total al sistema, el veterinario tiene acceso exclusivo a los módulos de historial clínico, consultas y recetas, y el trabajador accede a los módulos operativos como registro de clientes y pacientes, gestión de citas, inventario y ventas.

En cuanto al cifrado, las contraseñas se almacenan con bcrypt, los JWTs se firman con ES256 y las claves se almacenan en la tabla jwks de PostgreSQL rotándose periódicamente, y las comunicaciones utilizan TLS/SSL entre todos los servicios. La defensa contra ataques web se aborda mediante consultas parametrizadas con Drizzle ORM y Hibernate Reactive para prevenir inyección SQL, escape automático con Astro y React contra XSS, protección CSRF mediante cookies SameSite y tokens CSRF, y configuración restrictiva de CORS.

Requerimientos de disponibilidad y rendimiento

El sistema debe cumplir con condiciones que aseguren su operatividad continua. Se requiere disponibilidad durante la jornada laboral con monitoreo mediante endpoints de health check en cada servicio. Las consultas de historial clínico deben responder en menos de 500ms y la búsqueda por QR debe ser instantánea. El procesamiento de datos debe ser eficiente para reducir los tiempos de espera en la atención. La replicación en streaming de PostgreSQL garantiza la disponibilidad de los datos, y los backups completos semanales mediante pg_dump con archivado continuo de WALs permiten la recuperación point-in-time.

Interoperabilidad con sistemas externos

El sistema debe estar preparado para interactuar con otros sistemas en el futuro, permitiendo la integración con plataformas externas de pago, redes sociales u otras

aplicaciones. Desde el enfoque SOA, esto implica que los servicios deben diseñarse de manera estandarizada para facilitar su reutilización e integración.

La documentación técnica de los endpoints de la API está disponible automáticamente mediante esquemas OpenAPI expuestos por cada servicio. A partir de estos esquemas, el cliente API se genera automáticamente utilizando `@hey-api/openapi-ts`, lo que garantiza que los tipos TypeScript y las funciones SDK estén siempre sincronizados con los servicios.

En esta primera etapa no se contempla la integración con plataformas externas de pago, redes sociales u otros sistemas de terceros, pero la arquitectura está preparada para ello.

Gobernanza y roles

Identificación de stakeholders

En la clínica veterinaria se identifican diversos stakeholders. La jefa veterinaria o responsable del negocio es la principal encargada de la atención de los pacientes, la toma de decisiones y la gestión general de la clínica. Los clientes o propietarios de mascotas demandan los servicios veterinarios y adquieren productos para el cuidado de sus mascotas. Los pacientes o mascotas constituyen el eje central del servicio, ya que sobre ellos se realiza la atención médica y el registro clínico. El personal de apoyo se considera como futuro equipo que podría participar en actividades como atención, registro de datos o gestión de inventario. El administrador del sistema es responsable de gestionar el sistema, mantener la integridad de los datos y supervisar el correcto funcionamiento de los servicios.

Roles y responsabilidades

En función de los stakeholders identificados se establecen los siguientes roles dentro del sistema. El administrador es el usuario con privilegios totales, encargado de crear y gestionar las cuentas de veterinarios y trabajadores, con acceso a todos los módulos sin restricciones y capacidad de modificar la configuración del sistema, sin participar directamente en la operación diaria de la clínica. El veterinario está a cargo de la atención clínica, gestiona el historial médico de los pacientes, emite diagnósticos, prescribe recetas y realiza los seguimientos correspondientes, con acceso exclusivo a los módulos de historial clínico, consultas y recetas, pero sin capacidad de gestionar usuarios ni modificar la configuración general. El trabajador cumple una función administrativa dentro del sistema, gestionando citas, registrando clientes y mascotas, y administrando el inventario y las ventas, con acceso a los módulos operativos y restricciones en el historial clínico y la administración de usuarios. El cliente es el propietario de la mascota y puede ser una persona registrada en el sistema sin ser necesariamente un usuario del mismo, pudiendo solicitar servicios veterinarios, agendar citas y recibir atención y seguimiento. El sistema a través de los servicios SOA se encarga de procesar solicitudes de información, validar datos ingresados,

generar códigos QR con protocolo sanipatitas://patient/{uuid} para mascotas, ejecutar operaciones de registro, consulta y actualización, y garantizar la disponibilidad de los servicios.

Reglas de negocio

Las reglas de negocio establecen las condiciones que deben cumplirse para garantizar el correcto funcionamiento de los procesos dentro de la veterinaria. Un paciente debe estar registrado antes de ser atendido y cada paciente debe estar asociado a un cliente o propietario. No se puede registrar una cita sin seleccionar previamente un paciente y un cliente, y las citas no deben superponerse en el mismo horario. El historial clínico debe actualizarse después de cada atención médica. Los productos deben contar con stock disponible para realizar una venta y no se puede vender sin registrar los datos del cliente. Solo usuarios autorizados pueden acceder al sistema mediante autenticación. Cada mascota registrada debe tener un código QR único generado automáticamente con el protocolo sanipatitas://patient/{uuid} que la identifique de forma unívoca en el sistema. El código QR de una mascota no puede ser reutilizado ni asignado a otro paciente, incluso si el paciente original es dado de baja.

Levantamiento de información (entrevistas)

Entrevista a la responsable del negocio

La entrevista fue realizada a la jefa veterinaria, quien desempeña el rol de responsable del negocio y usuaria operativa. A través de esta entrevista se logró identificar el funcionamiento actual de los procesos, así como las principales necesidades del negocio.

Entre los aspectos más relevantes identificados se encuentra que el proceso principal del negocio es la atención de pacientes, desde su registro hasta el tratamiento y seguimiento. La gestión de citas se realiza de manera manual, lo que dificulta la organización del tiempo, y el registro de pacientes con su historial clínico se mantiene en fichas y apuntes físicos, generando retrasos en la atención. Asimismo, el control de productos se realiza sin un sistema que permita conocer el stock en tiempo real, por lo que existe una necesidad evidente de contar con información organizada y accesible de manera inmediata.

Se evidenció que muchas actividades del negocio se repiten, como el registro de datos del cliente o del paciente, lo que representa una oportunidad para la implementación de servicios reutilizables.

La jefa veterinaria también manifestó su interés en contar con una forma rápida de identificar a las mascotas cuando llegan a la clínica, mencionando que en ocasiones pierde tiempo buscando el expediente de un paciente recurrente. Esta necesidad sustenta la incorporación del sistema de códigos QR con protocolo `sanipatitas://patient/{uuid}` para la búsqueda rápida de mascotas.

Análisis de resultados de la entrevista

A partir de la información obtenida se identificaron diversos hallazgos relevantes para el diseño de la arquitectura SOA. Se determinó que el negocio presenta un alto nivel de dependencia de procesos manuales, lo que genera ineficiencias en el flujo de trabajo. La ausencia de un sistema integrado ocasiona que la información no se encuentre disponible en tiempo real, afectando la toma de decisiones y la calidad del servicio.

Se identificaron funciones clave que pueden ser transformadas en servicios dentro de la arquitectura SOA, tales como el registro de pacientes, la gestión de citas, el control de inventario y el registro de ventas. Estas funciones son recurrentes y se relacionan entre sí, lo que las convierte en candidatas ideales para su reutilización.

Finalmente, se evidenció la necesidad de mejorar la organización de la información, reducir los tiempos de atención y optimizar los procesos del negocio mediante la automatización. Estos hallazgos sustentan la propuesta de implementar una arquitectura orientada a servicios que permita integrar las actividades de la veterinaria de manera eficiente, incluyendo un mecanismo de identificación rápida mediante códigos QR con protocolo sanipatitas://patient/{uuid}.

Identificación de servicios candidatos (SOA)

Funciones del negocio candidatas a servicios

A partir del análisis del funcionamiento de la clínica veterinaria se identificaron diversas funciones del negocio que pueden ser transformadas en servicios independientes. El registro de pacientes permite almacenar y gestionar la información básica de las mascotas, incluyendo la generación automática de un código QR con protocolo `sanipatitas://patient/{uuid}` para cada una. La gestión del historial clínico permite registrar, consultar y actualizar la información médica de los pacientes, siguiendo los recursos del estándar HL7 FHIR para condiciones, observaciones, vacunas y procedimientos. El registro y gestión de clientes almacena los datos de los propietarios de las mascotas. La gestión de citas permite programar, modificar y consultar citas médicas, evitando superposiciones de horarios. La gestión de productos e inventario controla el stock, registro y disponibilidad de productos, con alertas de reposición y vencimiento. El registro de ventas gestiona las transacciones realizadas y actualiza automáticamente el inventario. La gestión de recetas emite recetas médicas con productos, dosis e indicaciones de tratamiento. La búsqueda por QR permite la identificación rápida de mascotas mediante el escaneo de su código QR con protocolo `sanipatitas://patient/{uuid}`, redirigiendo automáticamente a su ficha de paciente.

Identificación de servicios compartidos

En este contexto se identifican los siguientes servicios compartidos que son consumidos por múltiples servicios de negocio. El Auth Service proporciona autenticación y validación de JWT a todos los demás servicios mediante endpoint JWKS. El servicio de validación de datos de cliente y paciente es utilizado por Patient Service, Appointment Service y Sales Service. El servicio de consulta de información de pacientes es reutilizado por MedicalRecord Service, Appointment Service. El servicio de consulta de productos es usado tanto por Inventory Service como por Sales Service. El servicio de disponibilidad de citas verifica horarios antes de que Appointment Service registre una nueva cita. El código QR de cada mascota se genera como funcionalidad del frontend utilizando el protocolo

sanipatitas://patient/{uuid}, y la URI se almacena en Patient Service durante el registro para su consulta durante el escaneo desde el frontend.

Definición preliminar de servicios

A partir de las funciones identificadas, se propone la siguiente definición preliminar de servicios dentro de la arquitectura SOA. El servicio de pacientes recibirá como entrada los datos del paciente (nombre, especie, edad, etc.) y devolverá la información registrada o actualizada del paciente. El servicio de historial clínico recibirá el ID del paciente y los datos médicos, y devolverá el historial clínico actualizado o consultado. El servicio de clientes recibirá los datos del cliente (nombre, contacto, etc.) y retornará el registro o consulta del cliente. El servicio de citas recibirá la fecha, hora, paciente y cliente, y generará una confirmación de cita o disponibilidad. El servicio de inventario recibirá los datos del producto (nombre, cantidad, precio) y devolverá el estado del stock actualizado. El servicio de ventas recibirá los datos de productos, cliente y cantidades, y producirá el registro de la venta con el total generado. Finalmente, el servicio de autenticación recibirá las credenciales del usuario y otorgará acceso permitido o denegado.

Diseño de inventario de servicios

Los servicios son la unidad fundamental de una arquitectura SOA. Cada servicio expone una interfaz que define su funcionalidad y una implementación que la encapsula, permitiendo que los consumidores interactúen con él sin conocer sus detalles internos. A partir del análisis de las funciones del negocio, se definió el siguiente inventario de servicios para la Clínica Veterinaria HC.

Tabla 14

Inventario de servicios del sistema

Servicio	Interfaz REST	Implementación
Auth Service	POST /login + GET /.well-known/ jwks.json	Bun + Elysia + Better Auth
Patient Service	CRUD /patients + CRUD / customers	Java 25 + Quarkus 3.35.2
Appointment Service	CRUD /appointments	Java 25 + Quarkus 3.35.2
MedicalRecord Service	CRUD /medical-records por paciente	Java 25 + Quarkus 3.35.2
Inventory Service	CRUD /products + alertas /alerts	Java 25 + Quarkus 3.35.2
Sales Service	POST /sales + GET /sales/{id}	Java 25 + Quarkus 3.35.2
Prescription Service	POST /prescriptions + GET / prescriptions/{id}	Java 25 + Quarkus 3.35.2

La orientación a servicios produce tres resultados directos en el contexto de esta clínica. El primero es la reutilización: Patient Service es consumido por Appointment Service, MedicalRecord Service y Sales Service, eliminando la duplicación de la lógica de registro de pacientes que hoy existe en cuadernos y fichas separadas. El segundo es la interoperabilidad: cada servicio expone su esquema OpenAPI, lo que permite al frontend generar un cliente TypeScript sincronizado con los contratos de los servicios. El tercero es la escalabilidad: al estar encapsulados en contenedores Docker con healthchecks, los servicios pueden escalarse de forma independiente según la demanda de cada dominio.

El diseño de cada servicio sigue el principio de atomicidad funcional. Auth Service gestiona exclusivamente autenticación y autorización, Patient Service se limita al registro y consulta de pacientes y clientes. Ninguno de los servicios listados puede descomponerse en servicios más pequeños sin perder cohesión. Las interfaces RESTful se definieron antes que las implementaciones, siguiendo el contrato primero, de modo que cada servicio pueda sustituirse sin afectar a sus consumidores.

Arquitectura empresarial y capas SOA

El sistema se organiza en cuatro capas que separan las responsabilidades del negocio, la lógica de aplicación, el almacenamiento de datos y la infraestructura tecnológica. Esta separación permite que cada capa evolucione de forma independiente.

Capa de negocio

La capa de negocio define la estrategia, los procesos y las reglas que gobiernan la operación de la clínica. El proceso principal es la atención integral del paciente veterinario, que comprende desde la recepción del cliente hasta la facturación del servicio. Las reglas de negocio incluyen que todo paciente debe estar registrado antes de ser atendido, que cada paciente debe estar asociado a un cliente propietario, y que las citas no deben superponerse en el mismo horario. Estas reglas se implementan en los servicios de aplicación, pero su definición, es decir, quién puede hacer qué, en qué orden y bajo qué condiciones, pertenece a esta capa.

Capa de aplicación

La capa de aplicación contiene los servicios SOA que ejecutan la lógica del negocio. Está compuesta por siete servicios backend (Auth, Patient, Appointment, MedicalRecord, Inventory, Sales y Prescription) y el frontend desarrollado en Astro y React. Cada servicio encapsula un dominio específico y se comunica con los demás mediante HTTP REST con autenticación JWT. El frontend orquesta las llamadas a los servicios según el flujo de atención, actuando como el punto de coordinación visible para el usuario.

La generación automática del cliente API a partir de los esquemas OpenAPI de cada servicio, mediante `@hey-api/openapi-ts`, garantiza que los tipos TypeScript y las funciones de llamada estén sincronizados con la interfaz real de cada servicio. Esto elimina la duplicación manual de definiciones y reduce los errores de integración.

Capa de datos

La capa de datos gestiona el almacenamiento persistente y la disponibilidad de la información del sistema. La base de datos principal es PostgreSQL 17, con un esquema

relacional que cubre ocho dominios: usuarios, clientes, pacientes, especies y razas, citas, historiales clínicos, productos, ventas y recetas. Cada servicio accede a los datos de su dominio a través de su propia capa de persistencia, y consulta información de otros dominios mediante llamadas REST al servicio correspondiente, no mediante acceso directo a tablas ajenas.

La replicación en streaming asíncrona mantiene una réplica en espera que puede promoverse a primario en caso de fallo del servidor principal. Las migraciones del esquema se gestionan con Drizzle Kit 0.31, que permite versionar y aplicar cambios de forma controlada. Los backups completos se realizan semanalmente mediante `pg_dump`, complementados con archivado continuo de WALs para recuperación point-in-time. Redis 7 se utiliza como almacén de sesiones activas y caché de consultas frecuentes, reduciendo la carga sobre la base de datos principal.

Capa de tecnología

La capa de tecnología comprende la infraestructura sobre la que se ejecutan los servicios. En el entorno de desarrollo, cada servicio se despliega como un contenedor Docker con un healthcheck que expone el endpoint `GET /health`. El archivo `docker-compose.yml` define las dependencias entre servicios y la configuración de red, verificando los healthchecks antes de iniciar servicios dependientes.

Para el entorno de producción se ha previsto el uso de k3s, una distribución ligera de Kubernetes que utiliza SQLite en lugar de etcd para el almacenamiento del estado del clúster. k3s puede ejecutarse en un solo nodo con requisitos mínimos de memoria y CPU (512 MB de RAM son suficientes), lo que lo hace adecuado para los recursos de cómputo limitados de la clínica. Cada servicio se despliega como un Deployment de Kubernetes con sondas de liveness y readiness que verifican el healthcheck del contenedor. k3s proporciona descubrimiento de servicios mediante DNS interno, balanceo de carga básico y reinicio automático de contenedores fallidos.

Composición de servicios y Enterprise Service Bus

La composición de servicios es la capacidad de ensamblar servicios atómicos para construir funcionalidades de nivel superior. En el sistema propuesto, los servicios atómicos son Auth Service, Patient Service, Inventory Service y Prescription Service: cada uno tiene una responsabilidad única y no divisible. Los servicios compuestos incluyen Appointment Service, que depende de Patient Service para validar pacientes y clientes, MedicalRecord Service, que consulta Patient Service para asociar el historial a un paciente, y Sales Service, que consume Inventory Service para verificar y descontar stock al registrar una venta.

La composición puede adoptar dos enfoques de coordinación. La coreografía distribuye la lógica entre todos los participantes: cada servicio conoce su rol y responde a eventos sin un coordinador central. La orquestación centraliza el control en un componente que dirige el flujo y mantiene el estado del proceso. Para este proyecto se eligió el enfoque de orquestación, con el frontend actuando como orquestador principal. La decisión responde a la naturaleza secuencial y transaccional de los procesos clínicos: una consulta veterinaria requiere que los pasos se ejecuten en orden estricto: verificar cita, actualizar historial, prescribir y vender. Ante un fallo en cualquier paso se debe poder notificar al usuario sin dejar el sistema en un estado inconsistente.

No se implementa un Enterprise Service Bus (ESB) como middleware central. Los servicios se comunican mediante HTTP REST con esquemas OpenAPI bien definidos y el número de servicios es manejable (siete servicios backend), un ESB agregaría complejidad innecesaria sin un beneficio proporcional. k3s cumple funciones equivalentes a las de un ESB en el plano de infraestructura, como enrutamiento, descubrimiento de servicios y balanceo de carga, sin acoplar la lógica de negocio al middleware de mensajería. Esta decisión mantiene la arquitectura ligera y alineada con los recursos disponibles de la clínica.

Orquestación de servicios en el sistema

La orquestación coordina la ejecución de múltiples servicios para completar un proceso de negocio de extremo a extremo. En el sistema de la Clínica Veterinaria HC, el frontend asume el rol de orquestador: recibe las solicitudes del usuario, determina la secuencia de llamadas a los servicios backend, mantiene el estado del flujo y maneja los errores que puedan ocurrir.

Flujo principal: atención integral del paciente

El proceso de atención integral orquesta seis servicios en una secuencia definida.

1. **Identificación del paciente.** El cliente llega a la clínica y el personal escanea el código QR de la mascota desde el frontend. La URI `sanipatitas://patient/{uuid}` se extrae y se consulta Patient Service para recuperar la ficha del paciente.
2. **Verificación de cita.** El frontend consulta Appointment Service para verificar si el paciente tiene una cita programada. Si no existe, se registra una nueva cita en el momento.
3. **Actualización del historial clínico.** Durante la consulta, el veterinario registra diagnósticos, tratamientos y observaciones. El frontend envía los datos a MedicalRecord Service, que los almacena siguiendo los recursos del estándar HL7 FHIR.
4. **Prescripción.** Si el veterinario receta medicamentos, el frontend llama a Prescription Service para generar la receta asociada al paciente.
5. **Venta de productos.** Si el cliente adquiere productos, el frontend invoca Sales Service para registrar la venta, que a su vez consume Inventory Service para descontar el stock.
6. **Confirmación.** El frontend muestra un resumen de la atención realizada y ofrece la opción de imprimir o enviar el código QR actualizado del paciente.

Flujo secundario: control de inventario

El módulo de inventario opera con un flujo independiente pero conectado al principal. Cuando Sales Service registra una venta, Inventory Service evalúa si el stock de cada producto vendido ha alcanzado el nivel mínimo configurado. De ser así, el servicio activa una alerta de reposición que el frontend muestra al personal en el panel de control. Este flujo es asíncrono: no bloquea la transacción de venta, pero garantiza que el personal reciba la notificación en la siguiente interacción con el sistema.

Manejo de errores

Cada llamada del frontend a un servicio backend incluye manejo de errores en el cliente. Si un servicio no responde, por ejemplo, si Sales Service está caído tras haberse registrado el historial clínico, el frontend muestra un mensaje de error al usuario y registra el incidente sin modificar los datos ya persistidos. La consistencia del sistema se basa en la atomicidad de cada operación individual: cada servicio completa su transacción de forma independiente, y el frontend es responsable de coordinar el flujo general sin mantener estado transaccional distribuido.

Despliegue orquestado con Docker y k3s

La infraestructura de despliegue replica el patrón de orquestación a nivel de operaciones. Cada servicio backend define un `Dockerfile` con un healthcheck que verifica el endpoint `GET /health`. En el entorno de desarrollo, Docker Compose levanta los servicios en el orden definido por sus dependencias, verificando los healthchecks antes de iniciar servicios dependientes.

Para el entorno de producción, k3s gestiona el ciclo de vida de los contenedores. Cada servicio se despliega como un Deployment con sondas de liveness y readiness. La sonda de liveness verifica que el servicio responda cada 10 segundos, si falla tres veces consecutivas, k3s reinicia el contenedor. La sonda de readiness verifica que el servicio esté listo para recibir tráfico antes de enviarle solicitudes. k3s proporciona descubrimiento de servicios mediante DNS interno bajo el namespace del servicio, distribuye el tráfico entre las réplicas de cada

servicio y aplica las configuraciones de recursos (memoria, CPU) definidas en el manifiesto. Todo el clúster se describe en archivos YAML versionados, lo que permite reproducir el entorno de producción en cualquier máquina que ejecute k3s, incluyendo el servidor local de la clínica.

Implementación Backend

Servicios del sistema

Tabla 15

Inventario de servicios del sistema

Servicio	Interfaz REST	Implementación
Auth Service	POST /sign-in/email + GET / jwks + admin CRUD	Bun + Elysia + Better Auth
Patient Service	CRUD /patient + /client + / species + /breed	Java 25 + Quarkus 3.37.0 + Hibernate Reactive
Appointment Service	CRUD /appointment + GET / appointment/events (SSE)	Java 25 + Quarkus 3.37.0 + Hibernate Reactive

Los tres servicios comparten librerías comunes, lo que garantiza consistencia en modelos de dominio, validaciones y middleware transversal.

API REST y base de datos

Cada servicio expone una API REST documentada con OpenAPI 3.0. Las especificaciones se generan automáticamente: SmallRye OpenAPI en Quarkus y Better Auth en Elysia.

Tabla 16

Endpoints principales por servicio

Servicio	Endpoint	Roles
Auth	POST /api/auth/sign-in/email	público
Auth	POST /api/auth/sign-up/email	público
Auth	GET /api/auth/jwks	público
Auth	POST /api/auth/token	público
Auth	GET /api/auth/session	autenticado
Auth	*/api/auth/admin/*	admin
Patient	GET /api/patient	admin, veterinarian, worker
Patient	POST /api/patient	admin, worker
Patient	PUT /api/patient/{id}	admin, worker
Patient	DELETE /api/patient/{id}	admin
Appointment	GET /api/appointment	admin, veterinarian, worker
Appointment	POST /api/appointment	admin, veterinarian, worker

Servicio	Endpoint	Roles
Appointment	PUT /api/appointment/{id}	admin, veterinarian, worker
Appointment	DELETE /api/appointment/{id}	admin
Appointment	GET /api/appointment/events	admin, veterinarian, worker

La base de datos es **PostgreSQL 17** con soporte para texto completo en español mediante tsvector y claves primarias UUID v7 generadas con `uuid_generate_v7()`.

Comunicación entre servicios

El balanceador HAProxy expone un único punto de entrada y enruta según el prefijo de la ruta:

1	/api/auth	-> auth:3000	 Text
2	/api/patient	-> patient:8080	
3	/api/appointment	-> appointment:8081	
4	/*	-> frontend:1420	

La comunicación inter-servicios se realiza mediante HTTP con propagación de token JWT. Cada servicio valida el token de forma autónoma consultando el endpoint JWKS del Auth Service, lo que elimina la dependencia de un punto central de validación en cada petición.

Integración Frontend

Stack del cliente

El frontend está construido con **Astro 6** (SSR con adaptador Node.js) y **React 19** como librería de componentes. La interfaz se distribuye como aplicación web progresiva (PWA) y como aplicación de escritorio mediante **Tauri** (Rust).

Tabla 17

Stack tecnológico del frontend

Capa	Tecnología
Routing	TanStack Router con code-splitting por archivo
Estilos	Tailwind CSS v4 + @base-ui/react + @sanipatitas/ui
Estado	TanStack React Query + nanostores
Formularios	React Hook Form + Zod

Capa	Tecnología
Tablas	TanStack React Table
Cliente API	@hey-api/openapi-ts generado desde OpenAPI
Iconos	React Icons (Tabler), SVG vía vite-plugin-svgr
Notificaciones	Sonner (toasts)
Temas	next-themes (claro/oscurο)
PWA	@vite-pwa/astro con Workbox

Consumo de servicios mediante cliente generado

La librería @hey-api/openapi-ts genera automáticamente tipos TypeScript, funciones de llamada HTTP y hooks de TanStack React Query a partir de los esquemas OpenAPI de cada servicio. Esto elimina la duplicación manual de definiciones y garantiza que el frontend esté sincronizado con la interfaz real del backend.

Cada servicio expone su especificación OpenAPI en un endpoint dedicado:

1. Auth: GET /api/auth/openapi.json
2. Patient: GET /api/patient/openapi.json
3. Appointment: GET /api/appointment/openapi.json

Un script de build (generate-api-client) consulta los tres endpoints y @hey-api/openapi-ts produce:

1. sdk.gen.ts — funciones fetch tipadas
2. types.gen.ts — interfaces TypeScript para request/response
3. zod.gen.ts — esquemas Zod para validación en runtime
4. @tanstack/react-query.gen.ts — hooks useQuery y useMutation

Cada petición al backend incluye el token JWT en el header Authorization:

Bearer {token}. El token se almacena en una cookie httpOnly

(sanipatitas.session_token) para mitigar ataques XSS. Si un servicio responde con código 401, el frontend redirige al usuario a la pantalla de inicio de sesión.

Flujos de usuario principales

Identificación por QR. El personal escanea el código QR de la mascota generado con el protocolo `sanipatitas://patient/{uuid}`. La aplicación extrae el UUID, consulta `GET /api/patient/{uuid}` y muestra la ficha completa del paciente en una hoja lateral.

Gestión de citas. El trabajador selecciona paciente y cliente, verifica disponibilidad de horarios y registra la cita. El Appointment Service publica un evento SSE que actualiza la vista de todos los usuarios conectados en tiempo real.

Consulta de historial. El veterinario accede a la ficha del paciente y consulta el historial de citas registradas. Las tablas de auditoría de Hibernate Envers (`patient_aud`, `appointment_aud`) permiten reconstruir el historial completo de cambios sobre cada registro.

Administración de Procesos y Auditoría

Servicio transversal de auditoría

El sistema incorpora un servicio transversal de auditoría orientado al registro de transacciones críticas. Su objetivo es dejar evidencia estructurada de las operaciones sensibles ejecutadas en la plataforma, registrando como mínimo tres datos principales: **quién** realizó la acción, **cuándo** ocurrió y **qué** operación fue ejecutada.

Se consideran críticas las peticiones POST, PUT, PATCH y DELETE sobre los módulos de pacientes, clientes, especies, razas, citas y autenticación.

Cada evento de auditoría registra:

1. `type: "audit"` para distinguir eventos de auditoría de otros logs
2. `who:` usuario autenticado, extraído del claim sub del JWT
3. `role:` rol del usuario (`admin`, `veterinarian`, `worker`)
4. `when:` timestamp ISO-8601 con precisión de milisegundos
5. `service:` servicio de origen (`patient`, `appointment`, `auth`)
6. `action:` tipo de operación (`CREATE`, `UPDATE`, `DELETE`)
7. `resourceType:` tipo de recurso (`patient`, `client`, `appointment`, etc.)

8. `resourceId`: identificador UUID del recurso afectado
9. `method` y `path`: método y ruta HTTP de la petición
10. `status`: código de respuesta HTTP
11. `durationMs`: tiempo de procesamiento en milisegundos
12. `correlationId`: identificador único para trazabilidad entre servicios

La implementación se realiza en tres capas complementarias.

Logs estructurados en tiempo real.

En los servicios Quarkus, `AuditLogFilter` es un filtro HTTP transversal con prioridad `USER` que intercepta cada petición y genera un log JSON plano con `ObjectMapper`. El filtro emite a través del logger `sanipatitas.audit`.

En el servicio de autenticación, `auditMiddleware` (Elysia) cumple la misma función para las operaciones de escritura del flujo de autenticación. Ambos mecanismos producen registros con exactamente el mismo esquema de campos.

Hibernate Envers (histórico en base de datos).

Se integra `quarkus-hibernate-envers` sobre una unidad de persistencia JDBC dedicada, separada de la unidad reactiva principal. Cada entidad de negocio está anotada con `@Audited`. Envers genera automáticamente tablas de auditoría con el sufijo `_AUD` que almacenan el estado anterior y posterior de cada operación de escritura.

`SanipatitasRevisionEntity` extiende `DefaultRevisionEntity` con los campos `username` y `role`. `SanipatitasRevisionListener` captura el usuario y rol del JWT mediante inyección CDI y los persiste junto con cada revisión. Las tablas generadas son:

Tabla 18

Tablas de auditoría generadas por Hibernate Envers

Tabla	Propósito
<code>revinfo</code>	Metadata de cada revisión: <code>username</code> , <code>role</code> , <code>timestamp</code>
<code>patient_aud</code>	Snapshots históricos de pacientes
<code>client_aud</code>	Snapshots históricos de clientes

Tabla	Propósito
appointment_aud	Snapshots históricos de citas
breed_aud	Snapshots históricos de razas
species_aud	Snapshots históricos de especies
user_aud	Snapshots históricos de usuarios

El mapeo XML de Envers se habilita mediante la propiedad `hibernate.xml_mapping_enabled=true` requerida por Hibernate 7.x.

Centralización en Loki y visualización en Grafana.

Los servicios envían logs directamente a **Loki** mediante HTTP, sin depender de plugins de Docker ni sidecars:

Tabla 19

Mecanismos de envío de logs a Loki por servicio

Servicio	Mecanismo
Auth (Elysia/Bun)	loki-transport.ts (Pino)
Patient (Quarkus)	LokiLogHandler (JUL)
Appointment (Quarkus)	LokiLogHandler (JUL)

LokiLogHandler es un handler de `java.util.logging` que se registra automáticamente en el root logger mediante `LokiLogSetup (@Observes @Initialized(ApplicationScoped.class))`. Bufferea los registros en lotes de 100 y los envía de forma asíncrona con `java.net.http.HttpClient`. Si la variable `LOKI_ENABLED` no es `true`, el handler no se activa.

Variables de entorno requeridas por cada servicio:

```
1 LOKI_ENABLED=true
2 LOKI_URL=http://observability-loki:3100
3 SERVICE_NAME=patient # o auth, appointment
```

YAML

Los logs de auditoría se almacenan en Loki con labels de stream `{service, level, environment}`. Los dashboards de Grafana se aprovisionan automáticamente. Las queries se ejecutan sobre Loki mediante LogQL:

```
1 {service=~"patient|appointment|auth"} | json | type="audit" logql
```

Modelado BPMN con Camunda 8

Se utilizó **Camunda 8** con **BPMN 2.0** para modelar los flujos de negocio del sistema.

Se modelaron cinco procesos: auditoría de transacción crítica, aprobación de cita veterinaria, alerta de inventario por venta, registro de paciente y flujo de autenticación y autorización.

Figura 3

Diagrama BPMN del flujo de auditoría de transacción crítica

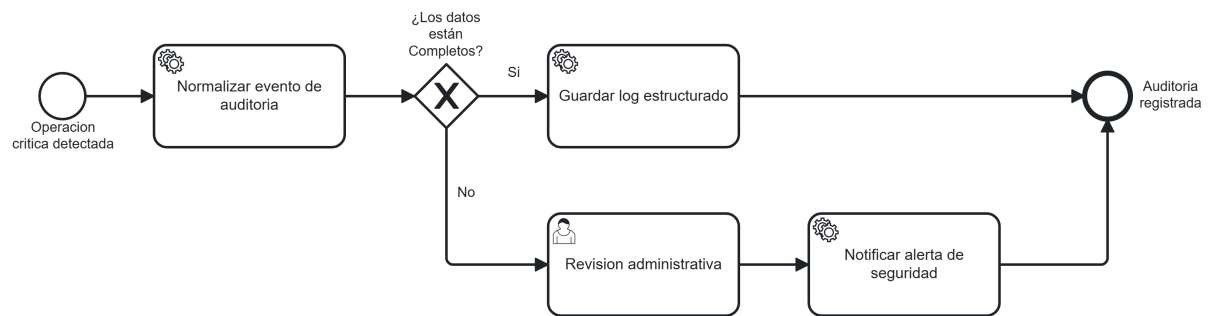


Figura 4

Diagrama BPMN del flujo de aprobación de cita veterinaria

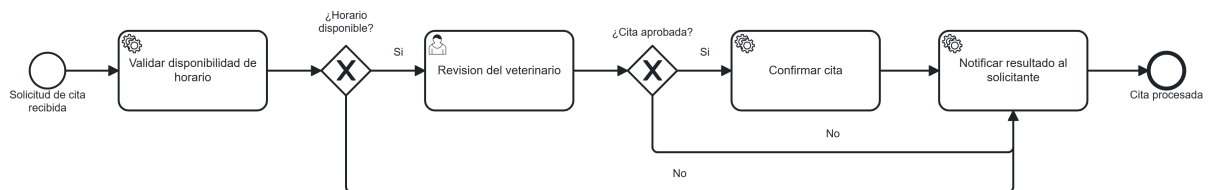


Figura 5

Diagrama BPMN del flujo de alerta de inventario por venta

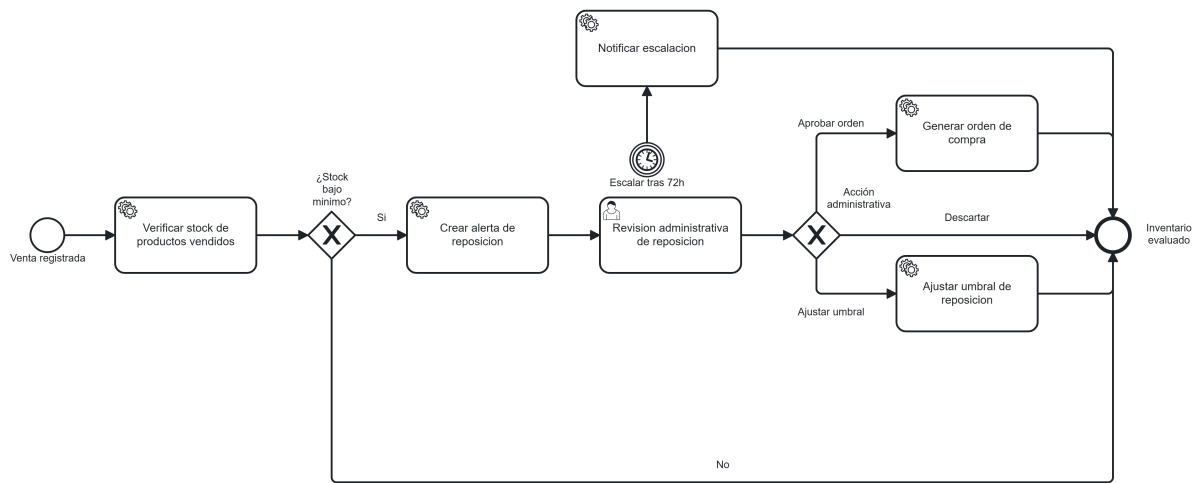
**Figura 6**

Diagrama BPMN del flujo de registro de paciente

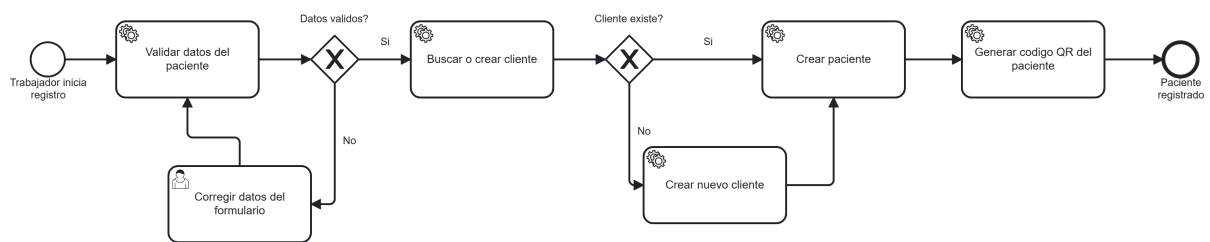
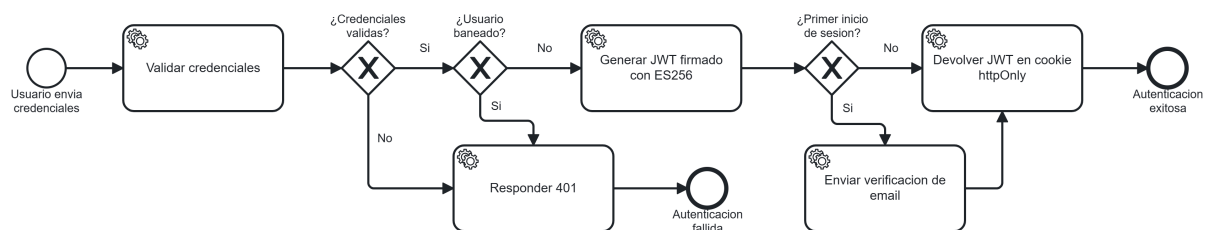
**Figura 7**

Diagrama BPMN del flujo de autenticación y autorización



Seguridad Implementada

Autenticación con JWT (ES256)

El sistema utiliza **JSON Web Tokens** firmados con el algoritmo **ES256** (curva elíptica P-256) para la autenticación entre el frontend y los servicios backend.

Flujo de autenticación:

1. El usuario envía email y password a POST /api/auth/sign-in/email
2. Better Auth valida las credenciales contra la tabla user de PostgreSQL (hash bcrypt)
3. Si son correctas, se crea una sesión y se almacena en una cookie httpOnly
4. Better Auth genera un JWT firmado con la clave privada ES256 almacenada en la tabla jwks. El token incluye los claims:
 - sub: identificador UUID del usuario
 - role: rol asignado (admin, veterinarian, worker)
 - iss: emisor
 - aud: audiencia
 - exp: fecha de expiración
 - iat: fecha de emisión
5. El token JWT se envía en cada petición mediante el header Authorization:


```
Bearer {token}
```

Validación autónoma de tokens:

Cada servicio Quarkus valida los JWT sin depender del Auth Service en cada petición. El endpoint JWKS (GET /api/auth/jwks) expone la clave pública. La configuración es:

```
1 mp.jwt.verify.publickey.algorithm=ES256
2 mp.jwt.verify.publickey.location=http://localhost/api/auth/jwks
3 smallrye.jwt.claims.groups=role
```

MicroProfile JWT obtiene la clave pública, verifica la firma del token y expone los claims a través del bean @Inject JsonWebToken. Los filtros TracingFilter y AuditLogFilter leen el sub y role para asignar who y role en los registros de auditoría y en el MDC de SLF4J.

Roles y autorización

El sistema define tres roles con permisos diferenciados, aplicados mediante la anotación `@RolesAllowed` de Jakarta Security:

Tabla 20

Roles del sistema y sus permisos

Rol	Permisos	Módulos
Administrador (admin)	Acceso total: crear, leer, actualizar, eliminar	Usuarios, pacientes, clientes, especies, razas, citas
Veterinario (veterinarian)	Acceso clínico: leer, crear, actualizar	Historial médico, citas
Trabajador (worker)	Acceso operativo: leer, crear, actualizar	Clientes, pacientes, citas

La autorización se verifica en dos capas:

1. **Controladores REST:** `@RolesAllowed` en cada método del controlador define los roles permitidos. DELETE solo está disponible para admin; GET está disponible para los tres roles.
2. **Filtro de auditoría:** `TracingFilter` (prioridad `AUTHENTICATION + 100`) corre después de la autenticación y extrae who y role para propagarlos en `X-Correlation-Id` y en el contexto de logging (MDC).

Si un usuario intenta acceder a un endpoint sin el rol requerido, el servicio responde con 403. Si el token es inválido o expiró, responde con 401.

Propagación del token entre servicios

Para las llamadas internas entre servicios se utiliza el mismo token JWT del usuario, propagado mediante el header `Authorization`. El `correlationId` generado por `TracingFilter` en el primer servicio se propaga al segundo mediante el header `X-Correlation-Id`, permitiendo correlacionar todos los eventos de una misma operación en Loki y Grafana.

Seguridad en el frontend

1. El token JWT se almacena exclusivamente en una cookie `httpOnly` con los flags `Secure` y `SameSite=None` para mitigar ataques XSS y CSRF.
2. Better Auth en el frontend gestiona automáticamente la renovación del token antes de que expire.
3. La configuración de CORS en el Auth Service restringe los orígenes permitidos a `tauri://localhost`, `http://tauri.localhost` y `http://localhost:1420`.
4. Las sesiones se almacenan en Redis con el prefijo `sanipatitas:auth:` para invalidación centralizada y revocación por parte del administrador.

Conclusiones

El análisis del negocio permitió evidenciar que la Clínica Veterinaria HC opera actualmente bajo un modelo basado en procesos manuales, lo cual genera limitaciones en la gestión de la información, retrasos en la atención y dependencia de una sola persona para la ejecución de múltiples funciones. Esta situación afecta directamente la eficiencia operativa y la calidad del servicio brindado.

Asimismo, se identificó la ausencia de un sistema tecnológico integrado que permita centralizar los datos relacionados con pacientes, clientes, citas, productos y ventas. La falta de integración dificulta el acceso oportuno a la información y limita la toma de decisiones, evidenciando la necesidad de implementar una solución estructurada.

A partir del levantamiento de información, se logró determinar que las principales funciones del negocio son repetitivas y manejan datos estructurados, lo que las convierte en candidatas ideales para ser transformadas en servicios dentro de una arquitectura orientada a servicios. Entre estas funciones destacan la gestión de pacientes, citas, historial clínico, inventario, ventas y recetas.

La división del sistema en servicios independientes como Auth Service, Patient Service, Appointment Service, MedicalRecord Service, Inventory Service, Sales Service y Prescription Service permite una mayor escalabilidad, mantenimiento independiente de cada dominio y reutilización de funcionalidades, principios fundamentales de SOA.

La incorporación del sistema de códigos QR con protocolo `sanipatitas://patient/{uuid}` para la búsqueda rápida de mascotas representa una mejora significativa en el flujo de trabajo de la clínica, permitiendo identificar a cada paciente de forma inmediata mediante el escaneo de su código QR único, reduciendo los tiempos de búsqueda y evitando errores de identificación. El protocolo personalizado garantiza que el sistema reconozca únicamente los códigos QR generados internamente.

Finalmente, se concluye que la implementación de una arquitectura SOA representa una alternativa viable para optimizar los procesos de la clínica veterinaria, permitiendo

sustituir los registros manuales por servicios digitales, mejorar la disponibilidad de la información, reducir los tiempos de respuesta y facilitar la escalabilidad del sistema en el futuro. Esta propuesta sienta las bases para el desarrollo de una solución tecnológica alineada con las necesidades reales del negocio.

Bibliografía

Desafíos del acceso a la información en salud: cuáles son y cómo resolverlos. (2023).

Conexia. <https://conexia.com/blog/desafios-del-acceso-a-la-informacion-en-salud-cuales-son-y-como-resolverlos/>

Digitalización en pausa: el 73% de pymes no sabe por dónde empezar su transformación

tecnológica. (2025). *El Comercio*. <https://elcomercio.pe/economia/peru/digitalizacion-en-pausa-el-73-de-pymes-no-sabe-por-donde-empezar-su-transformacion-tecnologica-noticia/>

Estrategias para una gestión eficiente de inventario en clínicas veterinarias. (2024). *provet*.

<https://www.provet.com/es/blog/gestion-inventario-eficiente-veterinaria>

Marks, N. (2024). Las mejoras en el estándar de atención gracias a la innovación en

veterinaria. *The Vetiverse*. <https://www.thevetiverse.com/es/ultimo/las-mejoras-en-el-estandar-de-atencion-gracias-a-la-innovacion-en-veterinaria/>

Ochoa, A. C., Murillo, A. C., & Rodas-Silva, J. (2021). El uso de aplicaciones Web para la Gestión de clínicas veterinarias y su incidencia en la mejora de procesos

administrativos. *Ecuadorian Science Journal*, 5(4), 109-120. <https://doi.org/10.46480/ESJ.5.4.174>